

Google Vision OCR Migration

Application Change Report

Application: DoGoodie Paired Testing Lab

Date: August 11, 2026

Implemented and verified locally - not deployed

Executive summary

The application OCR engine has been changed from server-executed Tesseract.js to Google Cloud Vision. The tester-facing workflow remains unchanged:

Select screenshot → Upload → OCR processing → Review detected boxes → Confirm fare, time, and ride tier

Only the OCR execution layer changes. Instead of using Vercel CPU and memory to run several Tesseract recognition passes, Vercel sends the stored screenshot to Google Cloud Vision and receives detected text and bounding polygons. Existing application rules continue to parse, display, confirm, validate, and store the result.

Why the change was required

The original implementation performed multiple CPU- and memory-intensive Tesseract passes for every screenshot. Serverless cold starts, language-model loading, image preprocessing, and simultaneous tester uploads could cause long waits, memory pressure, function timeouts, and the generic message *“An unexpected response was received from the server.”*

Google Cloud Vision moves the expensive OCR computation to Google's managed infrastructure. Vercel remains responsible for authentication, evidence authorization, retrieving the screenshot, calling the API, interpreting the response, enforcing study rules, and saving the result.

Workflow comparison

Previous Tesseract workflow

1. Tester selected and uploaded a screenshot to Supabase Storage.
2. Vercel downloaded the screenshot.
3. Vercel initialized Tesseract and its English language model.
4. Vercel performed several OCR passes for the selected card, status bar, battery region, and layout.
5. OCR candidates were parsed and saved.
6. Tester reviewed and confirmed the boxes.

New Google Vision workflow

1. Tester selects and uploads a screenshot to Supabase Storage.
2. The OCR queue records and schedules it.
3. Vercel sends one text-detection request to Google Cloud Vision.
4. Google returns detected text and bounding polygons.
5. The application converts the response into its existing candidate format.
6. Existing fare, time, and ride-tier parsers evaluate the candidates.

7. The tester confirms the boxes; existing server and database rules enforce the assignment.

Completed application changes

Google Vision client integration

The official `@google-cloud/vision` Node.js client was added. Credentials are read only from server-side environment variables and are not included in the client bundle.

Bounding-polygon conversion

Google word annotations are grouped into ordered text lines. Polygon coordinates are normalized so the existing screenshot review modal can render selectable ride, fare, time, and battery candidates.

Existing parsing retained

The fare parser, status-bar time parser, battery parser, service resolver, quote-time resolution, assignment-service comparison, and confirmation checks remain in use.

Provider configuration

```
OCR_PROVIDER=google
GOOGLE_CLOUD_PROJECT_ID=
GOOGLE_CLOUD_CLIENT_EMAIL=
GOOGLE_CLOUD_PRIVATE_KEY=
```

Tesseract remains available only when explicitly configured with `OCR_PROVIDER=tesseract`. Production never silently falls back to Tesseract.

Queue, retries, and errors

The database-backed queue retains queued, processing, completed, and failed states; controlled concurrency; automatic retries; and stale-worker recovery. Missing credentials and Google API response errors become explicit job errors and use the existing retry lifecycle.

Mocked tests

Unit tests verify Google word-to-line grouping, ordering, malformed-annotation handling, and combined bounding coordinates without spending API quota.

Verification completed

- TypeScript typecheck passed
- ESLint passed
- 105 tests passed
- 19 test files passed
- Production build passed
- Live Vision connection passed
- Grab example returned 71 annotations
- No tester evidence used

The live verification used only the repository's non-sensitive example screenshot. No credential value was printed or exposed.

Security and privacy controls

- Keep the private key only in `.env.local` and encrypted Vercel environment variables.
- Never commit credentials or expose them with a `NEXT_PUBLIC_` prefix.
- Keep the service account least-privileged.

- Original screenshots remain private evidence in Supabase Storage.
- Ownership, evidence linkage, assignment validation, confirmation, and audit rules remain server-controlled.
- Privacy documentation should identify Google Cloud as an external screenshot-processing provider.

Production deployment checklist

1. Confirm Cloud Vision API and billing are enabled.
2. Confirm the dedicated service account and key are active.
3. Add `OCR_PROVIDER=google` and all three credential variables to Vercel Production.
4. Mark the private key sensitive; never use `NEXT_PUBLIC_`.
5. Apply the OCR queue migration if it has not been applied.
6. Deploy a new Vercel build.
7. Test one screenshot end to end.
8. Test two simultaneous testers.
9. Test five simultaneous pairs (10 screenshots).
10. Review runtime logs, Google usage, queue time, retries, and failures.
11. Configure Google Cloud budget and usage alerts.

Rollback and fallback

Setting `OCR_PROVIDER=tesseract` deliberately restores the earlier engine for controlled local or emergency testing. Automatic production fallback is disabled because it would make server resource use unpredictable.

If Google Vision is temporarily unavailable, the original screenshot remains stored and its persisted job can retry. A provider failure does not delete evidence.

Deployment status

Local implementation complete. No Vercel deployment and no remote database modification were performed as part of this application change.

Separate dependency-security finding

The production dependency audit reports existing advisories, including advisories affecting Next.js 16.2.10 and transitive packages. These were not introduced by the Google Vision client and were not upgraded as part of the OCR migration. Address and regression-test them as a separate maintenance task before final production release.